

nezha-deploy.sh

Revision: 2 Last modified: 2026-06-16T19:45:00Z

Overview

Durable deployment wrapper for the full HelixTranslate stack on the **nezha** host. It always invokes `podman-compose` with `--env-file .env.nezha`, fixing heavy-testing iteration-2 finding #3: a fresh `podman-compose` up previously interpolated `{VAR}` substitutions (ports, `JWT_SECRET`) from the default `.env` (which on nezha holds only `HELIX_RELEASE_PREFIX`) instead of `.env.nezha`, so ports/JWT silently fell back to their `:default` values unless the operator manually exported `.env.nezha` into the shell.

Prerequisites

- `podman + podman-compose >= 1.5.0` (the `--env-file` flag).
- `compose.nezha.yml` and a provisioned `.env.nezha` (gitignored; see `.env.nezha.example`) present in the project root.

Usage examples

```
# Fresh from-scratch boot of the whole stack (correct ports/JWT,
# no workaround):
bash scripts/nezha-deploy.sh up

# Rebuild + reboot a single service after a source fix:
bash scripts/nezha-deploy.sh reboot api-server
bash scripts/nezha-deploy.sh reboot server

# Rebuild + reboot the WHOLE stack onto the fresh image (no
# service arg):
# recreates ALL app services sharing the helixtranslate:nezha
# image
# (grpc-server, api-server, server, monitor-server) so none is
# stranded on
# the old image. postgres/redis (external images) are left
# running.
bash scripts/nezha-deploy.sh reboot

# Validate that interpolation resolves to the real ports (not
# defaults):
bash scripts/nezha-deploy.sh config | grep -E '18080|18443|50061|
18090'
```

Stack status / teardown:

```
bash scripts/nezha-deploy.sh ps
```

```
bash scripts/nezha-deploy.sh down
```

Edge cases

- Missing `compose.nezha.yml` or `.env.nezha` → script exits non-zero with an actionable message (never deploys with defaults silently).
- Unknown action → usage message + exit 2.
- **reboot with NO service arg (dependency-ordering fix, Rev 2):** because all four app services share the `helixtranslate:nezha` image tag and `podman-compose 1.5.0` will not recreate a same-tag running container, a no-arg reboot now explicitly `stop+podman rm`s the full app set (`grpc-server` `api-server` `server` `monitor-server`) before `up -d`, so the fresh image reaches every dependent. Previously the `stop+rm` loop was gated on a `named-service` arg, so a no-arg reboot rebuilt the image but stranded the already-running app containers (esp. `helixtranslate-monitor`) on the old binary — the §11.4.108 `SOURCE→ARTIFACT→RUNTIME` gap.

Internal behaviour

The single load-bearing line is the `COMPOSE` array: `podman-compose --env-file .env.nezha -f compose.nezha.yml`. Every action funnels through it, so no code path can deploy without `.env.nezha` driving interpolation.

Related scripts

- `scripts/nezha-entrypoint.sh` — per-container binary selector (`SERVICE` env).

Last verified

2026-06-16 — `podman-compose config` on `nezha` resolves the real 18080/18443/50061/18090 ports + the real `JWT_SECRET` via this wrapper.